

Resolución: Cadenas de bits a distancia de Hamming h usando Backtracking

Este documento presenta la resolución detallada del problema de diseño de un algoritmo de Backtracking (Vuelta Atrás) para generar todas las cadenas de bits de longitud n que tienen exactamente una distancia de Hamming h respecto a la cadena nula (es decir, cadenas con exactamente h unos).

a) Representación de Soluciones y Restricciones

Para resolver este problema mediante backtracking, primero debemos definir de manera formal cómo se estructuran nuestras soluciones candidatas y qué reglas deben cumplir para ser válidas.

1. Representación de las Soluciones

Representamos cada solución como una tupla de tamaño n :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i representa el valor del bit en la posición i .

2. Restricciones Explícitas

Las restricciones explícitas definen el dominio de cada variable de la tupla:

$$x_i \in \{0, 1\} \quad \forall i \in \{1, 2, \dots, n\}$$

Es decir, cada elemento de la secuencia solo puede tomar el valor de 0 o 1.

3. Restricciones Implícitas

Las restricciones implícitas determinan las condiciones que debe cumplir la tupla completa para ser considerada una solución válida del problema.

Dado que la distancia de Hamming respecto a la cadena nula $(0, 0, \dots, 0)$ es equivalente al número de bits establecidos en 1 (peso de Hamming), la restricción implícita exige que la suma de los componentes de la tupla sea exactamente h :

$$\sum_{i=1}^n x_i = h$$

b) Árbol del Espacio de Soluciones y su Tamaño

Descripción del Árbol

El espacio de búsqueda se representa mediante un **árbol de decisión binario**:

- **Raíz (Nivel 0)**: Representa el estado inicial antes de tomar ninguna decisión (ningún bit asignado).
- **Nivel i ($1 \leq i \leq n$)**: Representa la toma de decisión para la variable x_i .
- **Ramificación**: De cada nodo en el nivel $i - 1$ nacen dos ramas (hijos):
 - Una rama izquierda que asigna $x_i = 0$.
 - Una rama derecha que asigna $x_i = 1$.
- **Hojas (Nivel n)**: Representan asignaciones completas de la tupla $X = (x_1, \dots, x_n)$.

Tamaño del Árbol (Fuerza Bruta vs. Filtrado)

Si realizáramos una búsqueda por fuerza bruta sin ningún tipo de poda:

1. **Número de hojas (soluciones candidatas totales)**: 2^n
2. **Número total de nodos en el árbol**: $1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$

Sin embargo, el número real de **soluciones válidas** (hojas que cumplen la restricción implícita) es mucho menor y está determinado por el coeficiente binomial:

$$\binom{n}{h} = \frac{n!}{h!(n-h)!}$$

El objetivo del Backtracking con predicados acotadores es evitar explorar aquellas ramas que sabemos con certeza que no nos conducirán a una de estas $\binom{n}{h}$ soluciones.

c) Predicados Acotadores (Pruning)

Para evitar la exploración de ramas infructuosas, introducimos criterios de poda en cada etapa de la recursión.

Supongamos que nos encontramos en el nivel k del árbol (es decir, ya hemos tomado decisiones para $x_1, x_2, \dots, x_{k-1}$ y nos disponemos a elegir el valor de x_k). Definimos la variable acumuladora u como la cantidad de unos que ya hemos colocado en la tupla:

$$u = \sum_{j=1}^{k-1} x_j$$

1. Poda por Exceso de Unos

Si en el paso actual ya hemos colocado más de h unos, es imposible que cualquier extensión de esta tupla sea válida (ya que no podemos "quitar" unos en los niveles siguientes).

- **Condición de poda**: $u > h$

2. Poda por Defecto de Unos

Si el número de unos que llevamos acumulados (u), sumado a la cantidad máxima de bits que nos quedan por decidir desde la posición actual hasta el final ($n - k + 1$), es menor que el total de unos requeridos (h), nunca podremos llegar a tener exactamente h unos aunque asignemos todos los bits restantes como 1.

- Condición de poda: $u + (n - k + 1) < h$

3. Predicado Acotador de Viabilidad

Por lo tanto, la rama actual es viable y debe seguir explorándose si y solo si se cumple la siguiente condición:

$$u \leq h \quad \wedge \quad u + (n - k + 1) \geq h$$

d) Algoritmo en Pseudocódigo

```

Procedimiento GenerarCadenasHamming(n, h)
Inicio
  // inicializar la tupla solución con ceros
  solucion_actual <- Arreglo de tamaño n inicializado en [0, 0, ..., 0]
  total_soluciones <- 0

  // sub-procedimiento recursivo interno
  Procedimiento Backtracking(nivel, unos_acumulados)
  Inicio
    // 1. PREDICADOS ACOTADORES (PODAS)
    // Poda por exceso de unos
    Si unos_acumulados > h Entonces
      Retornar
    FinSi

    // Poda por defecto de unos (posiciones restantes en el arreglo)
    posiciones_restantes <- n - nivel
    Si unos_acumulados + posiciones_restantes < h Entonces
      Retornar
    FinSi

    // 2. CASO BASE (Se ha completado la tupla de tamaño n)
    Si nivel = n Entonces
      Imprimir(solucion_actual)
      total_soluciones <- total_soluciones + 1
      Retornar
    FinSi

    // 3. RAMIFICACIÓN (Exploración de alternativas)

    // Opción A: Intentar asignando 0 en el bit de la posición actual
    solucion_actual[nivel] <- 0
    Backtracking(nivel + 1, unos_acumulados)

    // Opción B: Intentar asignando 1 en el bit de la posición actual
    solucion_actual[nivel] <- 1
    Backtracking(nivel + 1, unos_acumulados + 1)

  FinProcedimiento

```

```
// Llamada inicial desde el nivel 0 y con 0 unos acumulados  
Backtracking(0, 0)  
Imprimir("Cantidad total de soluciones válidas: ", total_soluciones)  
FinProcedimiento
```

Análisis de la Complejidad con Podas

Gracias a las podas por exceso y por defecto, el algoritmo no explora las hojas inválidas. La complejidad temporal se reduce de un coste exponencial plano de $O(2^n)$ a un coste proporcional a la cantidad de soluciones reales generadas más las bifurcaciones internas viables, lo cual es asintóticamente óptimo:

$$O\left(\binom{n}{h}\right)$$